

Lab 2: ทำความสะอาดข้อมูลดิบจากเซนเซอร์ IoT

วัตถุประสงค์: ตรวจสอบและแก้ไขปัญหา Error, Duplicate, Missing Value และ Outlier ในไฟล์ `iot_sensor_raw_data_extended.csv` ซึ่งมีข้อมูลดิบทั้งหมด 480 แถว จากอุปกรณ์เซนเซอร์ 5 ตัว (SN-9982 ถึง SN-9986)

1. Data Quality Rules ที่กำหนด

Data Quality Rule	คำอธิบาย
Temperature Range	อุณหภูมิ (temp_c) ต้องอยู่ระหว่าง 0-50 องศาเซลเซียส ค่านอกช่วงนี้ (เช่น -999, 85.0) ถือเป็น Outlier
Device ID Not Null	device_id ห้ามเป็นค่าว่าง เนื่องจากเป็น Primary Key ที่ใช้ระบุอุปกรณ์
No Duplicate Records	ห้ามมีข้อมูลซ้ำที่มี device_id และ timestamp เดียวกัน (คีย์ผสมต้อง Unique)
Battery Range	ค่าแบตเตอรี่ (battery) ต้องอยู่ระหว่าง 0-100 (%) ค่านอกช่วงนี้ (เช่น -5, 999, 115) ถือเป็น Outlier
Motion Format	ค่าคอลัมน์ motion ต้องอยู่ในรูปแบบบูลีนที่รู้จัก (true/false/yes/no/0/1) ค่าอื่น เช่น 'unknown', 'TRUEE' ถือว่าไม่ถูกต้อง

2. รายงานปัญหาที่พบในข้อมูลดิบ

ประเภทปัญหา	จำนวนแถวที่พบ (จาก 480 แถว)	ตัวอย่างที่พบในไฟล์
Duplicate Rows (device_id+timestamp ซ้ำ)	50 แถว (25 คู่)	เช่น SN-9986 เวลา 14:51:00 ปรากฏซ้ำกัน 2 แถวเป๊ะ
Temperature Outlier (นอกช่วง 0-50°C)	10 แถว	ค่า -999.0 (เซนเซอร์ error) และค่า 85.0 (ผิดปกติ) ที่ SN-9983, SN-9984, SN-9985
Battery Outlier (นอกช่วง 0-100%)	5 แถว	ค่า -5, -1, 999, 115, 108 ซึ่งเป็นไปไม่ได้ทางกายภาพ
Missing Value: temp_c	7 แถว	ค่าว่าง (NaN) ในคอลัมน์อุณหภูมิ
Missing Value: battery	4 แถว	ค่าว่าง (NaN) ในคอลัมน์แบตเตอรี่
Missing Value: motion	4 แถว	ค่าว่าง (NaN) ในคอลัมน์การตรวจจับความเคลื่อนไหว
Missing Value: device_id	2 แถว	ไม่สามารถระบุอุปกรณ์ต้นทางได้ ต้องตรวจสอบย้อนกลับกับ Log ฮาร์ดแวร์
Invalid Motion Format	3 แถว	ค่าเช่น 'unknown', 'TRUEE', 'FALSEE' ซึ่งไม่ตรงกับรูปแบบมาตรฐาน

3. Source Code

สคริปต์นี้เป็น Code Python ที่ใช้ในการ Clean Raw data

```
###
=====
Lab 2
=====
###

import pandas as pd
import numpy as np

RAW_FILE = r"E:\angly\lab2\iot_sensor_raw_data_extended.csv"
OUT_CLEANED = r"E:\angly\lab2\cleaned_iot_sensor_data.csv"
OUT_REPORT = r"E:\angly\lab2\data_quality_report.csv"

# -----
# กำหนด Data Quality Rules
# -----
TEMP_MIN, TEMP_MAX = 0, 50      # อุณหภูมิต้องอยู่ระหว่าง 0-50 องศาเซลเซียส
```

BATTERY_MIN, BATTERY_MAX = 0, 100 # แบตเตอรี่ต้องอยู่ระหว่าง 0-100

โหลดข้อมูลดิบ

```
raw = pd.read_csv(RAW_FILE)
raw_row_count = len(raw)
print(f"[STEP 1] โหลดข้อมูลดิบสำเร็จ: {raw_row_count} แถว")
```

df = raw.copy() # คัดลอกสำรองไว้นะ

ตรวจสอบปัญหาคุณภาพข้อมูลก่อนแก้ไขเพื่อทำรายงาน

```
issues = pd.DataFrame(index=df.index)

issues["missing_device_id"] = df["device_id"].isna() | (df["device_id"].astype(str).strip() == "")
issues["missing_temp"] = df["temp_c"].isna()
issues["missing_motion"] = df["motion"].isna()
issues["missing_battery"] = df["battery"].isna()

issues["temp_out_of_range"] = df["temp_c"].notna() & ((df["temp_c"] < TEMP_MIN) | (df["temp_c"] > TEMP_MAX))
issues["battery_out_of_range"] = df["battery"].notna() & ((df["battery"] < BATTERY_MIN) | (df["battery"] > BATTERY_MAX))
issues["duplicate_row"] = df.duplicated(subset=["device_id", "timestamp"], keep=False)
motion_valid_raw = {"true", "false"}
issues["invalid_motion_format"] = df["motion"].notna() & (~df["motion"].astype(str).lower().isin(
    {"true", "false", "0", "1", "yes", "no"}))
```

```
summary_before = {
    "จำนวนแถวทั้งหมด (ก่อนทำความสะอาด)": raw_row_count,
    "device_id ว่าง": int(issues["missing_device_id"].sum()),
    "temp_c ขาดหาย (missing)": int(issues["missing_temp"].sum()),
    "motion ขาดหาย (missing)": int(issues["missing_motion"].sum()),
    "battery ขาดหาย (missing)": int(issues["missing_battery"].sum()),
    "temp_c ผิดช่วง (outlier, นอก 0-50)": int(issues["temp_out_of_range"].sum()),
    "battery ผิดช่วง (outlier, นอก 0-100)": int(issues["battery_out_of_range"].sum()),
    "แถวซ้ำ (duplicate device_id+timestamp)": int(issues["duplicate_row"].sum()),
    "motion รูปแบบไม่ถูกต้อง (เช่น TRUEE, unknown)": int(issues["invalid_motion_format"].sum()),
}
```

print(f"[STEP 2] สรุปปัญหาที่พบ (ก่อนทำความสะอาด):")

for k, v in summary_before.items():

print(f" {k}: {v}")

STEP 3: ทำความสะอาดข้อมูล

3.1 ลบข้อมูลซ้ำ (duplicate rows ที่มี device_id + timestamp เหมือนกันทุกค่า) -> เก็บแถวแรกไว้

before_dedup = len(df)

df = df.drop_duplicates(subset=["device_id", "timestamp"], keep="first").reset_index(drop=True)

removed_duplicates = before_dedup - len(df)

print(f"[STEP 3.1] ลบข้อมูลซ้ำ: ลบไป {removed_duplicates} แถว (เหลือ {len(df)} แถว)")

3.2 แปลงค่าผิดปกติ (outlier) ให้เป็น NULL (NaN) แทนที่จะลบทิ้งแถว

mask_temp_bad = df["temp_c"].notna() & ((df["temp_c"] < TEMP_MIN) | (df["temp_c"] > TEMP_MAX))

mask_batt_bad = df["battery"].notna() & ((df["battery"] < BATTERY_MIN) | (df["battery"] > BATTERY_MAX))

n_temp_nulled = int(mask_temp_bad.sum())

n_batt_nulled = int(mask_batt_bad.sum())

df.loc[mask_temp_bad, "temp_c"] = np.nan

df.loc[mask_batt_bad, "battery"] = np.nan

print(f"[STEP 3.2] แปลงค่าผิดปกติเป็น NULL: temp_c {n_temp_nulled} แถว, battery {n_batt_nulled} แถว")

3.3 ทำความสะอาดคอลัมน์ motion ให้เป็นรูปแบบมาตรฐาน (boolean True/False) ค่าที่แปลไม่ได้ -> NaN

motion_map = {

"true": True, "false": False,

"1": True, "0": False,

"yes": True, "no": False,

"truee": True, "falsee": False, # แก้ค่าสะกดผิด

```
}

def normalize_motion(v):
    if pd.isna(v):
        return np.nan

    key = str(v).strip().lower()
    return motion_map.get(key, np.nan) # ค่าที่ไม่รู้จัก เช่น "unknown" -> NaN

n_motion_before_unknown = df["motion"].notna().sum()
df["motion"] = df["motion"].apply(normalize_motion)

n_motion_nulled = int((df["motion"].isna()).sum() - (raw["motion"].isna().sum() - removed_duplicates * 0))
print(f"{'แปลงคอตัมน์ motion เป็น boolean มาตรฐาน (ค่าที่แปลงไม่ได้ -> NULL)"}")
```

3.4 จัดการ Missing Value

- device_id ว่า: ไม่สามารถค่าที่ถูกห้อยได้ -> คงไว้เป็น NULL และตั้งสถานะเป็น error (ไม่ impute)

- temp_c ว่า: เป็น NULL: เติมด้วยค่าเฉลี่ยเคลื่อนที่ (interpolate) แยกตาม device_id เรียงตามเวลา

- battery ว่า: เติมด้วยวิธี forward-fill/backward-fill แยกตาม device_id (แบบเคอร์รี่เปลี่ยนแปลงซ้ำ)

- motion ว่า: แปลงไม่ได้: เติมด้วยค่า mode (ค่าที่พบบ่อยที่สุด) ของอุปกรณ์นั้น ๆ

```
df = df.sort_values(["device_id", "timestamp"]).reset_index(drop=True)
```

```
df["temp_c_imputed"] = False
df["battery_imputed"] = False
df["motion_imputed"] = False
```

for dev_id, grp_idx in df.groupby("device_id", dropna=True).groups.items():

```
    idx = list(grp_idx)
    # temp_c: interpolate ตามลำดับเวลาของอุปกรณ์นั้น ๆ
    before = df.loc[idx, "temp_c"]
    interpolated = before.interpolate(method="linear", limit_direction="both").round(1)
    changed = before.isna() & interpolated.notna()
    df.loc[idx, "temp_c_imputed"] = changed
    df.loc[idx, "temp_c"] = interpolated
```

battery: forward-fill แล้ว backward-fill (แบบไม่เปลี่ยนเร็ว)

```
before_b = df.loc[idx, "battery"]
filled = before_b.fill().bfill()
changed_b = before_b.isna() & filled.notna()
df.loc[idx, "battery_imputed"] = changed_b
df.loc[idx, "battery"] = filled
```

motion: เติมด้วย mode ของอุปกรณ์นั้น

```
before_m = df.loc[idx, "motion"]
if before_m.notna().any():
    mode_val = before_m.mode(dropna=True).iloc[0]
else:
    mode_val = np.nan
changed_m = before_m.isna()
df.loc[idx, "motion_imputed"] = changed_m
df.loc[idx, "motion"] = before_m.fillna(mode_val)
```

```
n_temp_imputed = int(df["temp_c_imputed"].sum())
```

```
n_batt_imputed = int(df["battery_imputed"].sum())
```

```
n_motion_imputed = int(df["motion_imputed"].sum())
```

```
print(f"{'STEP 3.4} เติมค่า Missing Value: temp_c {n_temp_imputed} แถว, "
```

```
      {'battery {n_batt_imputed} แถว, motion {n_motion_imputed} แถว')"
```

```
print("      (device_id ที่ว่าง จะไม่ถูกหาค่า เนื่องจากไม่มีข้อมูลอ้างอิงเพียงพอ)")
```

```
# -----
```

STEP 4: เพิ่มคอตัมน์ data_quality_status

```
# -----
```

```
def build_status(row):
```

```
    flags = []
    if pd.isna(row["device_id"]) or str(row["device_id"]).strip() == "":
        flags.append("missing_device_id")
    if row["temp_c_imputed"]:
        flags.append("temp_imputed")
    if row["battery_imputed"]:
```

```
        flags.append("battery_imputed")

    if row["motion_imputed"]:
        flags.append("motion_imputed")

    if pd.isna(row["temp_c"]):
        flags.append("temp_still_missing")

    if pd.isna(row["battery"]):
        flags.append("battery_still_missing")

    if pd.isna(row["motion"]):
        flags.append("motion_still_missing")

    return "ok" if not flags else ""].join(flags)

df["data_quality_status"] = df.apply(build_status, axis=1)
df = df.drop(columns=["temp_c_imputed", "battery_imputed", "motion_imputed"])

# -----
# STEP 5: สรุปผลก่อนหลังและบันทึกไฟล์
# -----

after_row_count = len(df)

summary_after = {
    "จำนวนแถวทั้งหมด (หลังทำความสะอาด)": after_row_count,
    "แถวที่ถูกลบ (ซ้ำ)": removed_duplicates,
    "แถวสถานะ ok (ไม่มีปัญหา)": int((df["data_quality_status"] == "ok").sum()),
    "แถวที่เติมค่า temp_c": n_temp_imputed,
    "แถวที่เติมค่า battery": n_batt_imputed,
    "แถวที่เติมค่า motion": n_motion_imputed,
    "แถวที่ device_id ข้างว่าง (ต้องตรวจสอบ manual)": int(df["data_quality_status"].str.contains("missing_device_id").sum()),
}

print(f"๓[STEP 5] สรุปผลหลังทำความสะอาด:")
for k, v in summary_after.items():
    print(f"   - {k}: {v}")

print(f"๓[STEP 5] เปรียบเทียบจำนวนแถวก่อน-หลัง:")
print(f"๑ ก่อนทำความสะอาด : {raw_row_count} แถว")
print(f"๑ หลังทำความสะอาด : {after_row_count} แถว (ลบแถวซ้ำออก {removed_duplicates} แถว)")
print(f"๑ หมายเหตุ: ไม่มีการลบแถวเนื่องจาก outlier/missing value ออกจากชุดข้อมูล ")
print(f"๑— มีเพียงการแปลงเป็น NULL หรือเติมค่า (impute) และระบุสถานะไว้ในคอลัมน์ data_quality_status")

# บันทึกไฟล์
import os
os.makedirs("/mnt/user-data/outputs", exist_ok=True)
df.to_csv(OUT_CLEANED, index=False)

report_rows = []
for k, v in summary_before.items():
    report_rows.append({'ขั้นตอน': "ก่อนทำความสะอาด", "รายการ": k, "จำนวน": v})
for k, v in summary_after.items():
    report_rows.append({'ขั้นตอน': "หลังทำความสะอาด", "รายการ": k, "จำนวน": v})
report_df = pd.DataFrame(report_rows)
report_df.to_csv(OUT_REPORT, index=False, encoding="utf-8-sig")

print(f"๓บันทึกไฟล์ Cleaned Data ที่: {OUT_CLEANED}")
print(f"๓บันทึกไฟล์รายงานสรุปที่: {OUT_REPORT}")
```

4. ตารางสรุปเปรียบเทียบก่อน-หลังทำความสะอาด

รายการ	ก่อนทำความสะอาด	หลังทำความสะอาด
จำนวนแถวทั้งหมด	480	455
แถวซ้ำ (duplicate)	50 แถว (25 คู่)	0 (ลบออกแล้ว 25 แถว)
temp_c ผิดช่วง/Outlier	10 แถว	0 (แปลงเป็น NULL แล้วเติมค่าใหม่ 15 แถว รวมของเดิมที่ขาดหาย)
battery ผิดช่วง/Outlier	5 แถว	0 (แปลงเป็น NULL แล้วเติมค่าใหม่ 8 แถว รวมของเดิมที่ขาดหาย)
motion รูปแบบไม่ถูกต้อง	3 แถว	0 (แปลงเป็น NULL แล้วเติมค่าด้วย mode 5 แถว)
device_id ว่าง	2 แถว	2 แถว (คงไว้ตามเดิม ระบุสถานะ missing_device_id ให้ตรวจสอบ manual)
แถวสถานะ data_quality_status = ok	-	425 แถว จาก 455 แถว (93.4%)

5. เหตุผลที่ไม่ควรลบ Raw Data ดัชนี

- Traceability / Audit Trail: หากพบข้อผิดพลาดในขั้นตอนทำความสะอาดภายหลัง จำเป็นต้องย้อนกลับไปตรวจสอบข้อมูลต้นฉบับเพื่อยืนยันความถูกต้อง
- การเปลี่ยนแปลง Business Rule: กฎการทำความสะอาด (เช่น ช่วงอุณหภูมิที่ยอมรับได้) อาจเปลี่ยนแปลงในอนาคต หากไม่มี Raw Data จะไม่สามารถประมวลผลใหม่ได้
- การวิเคราะห์คุณภาพเซนเซอร์: ข้อมูล Error/Outlier ที่ถูกคัดออกยังมีคุณค่าในการวิเคราะห์ว่าเซนเซอร์ตัวใดมีแนวโน้มเสียหรือทำงานผิดปกติบ่อย
- ข้อกำหนดด้านกฎหมาย/มาตรฐาน: หลายองค์กรมีข้อกำหนดให้เก็บข้อมูลดิบไว้เป็นหลักฐาน (Data Governance / Compliance)
- ป้องกันการสูญเสียข้อมูลถาวร: การ Impute หรือแปลงค่าเป็น NULL คือการประมาณค่า ไม่ใช่ค่าจริง หาก Raw Data หายไป จะไม่สามารถกู้คืนค่าจริงหรือทำวิธีทำความสะอาดแบบอื่นได้อีก